Universidade do Minho

AGENTIF AI

Síntese De Voz Em Tempo Real

Projeto de Informática



Beatriz Monteiro, Daniel Du, Daniel Furtado, Moisés Antunes, Pedro Gomes e Telmo Maciel

Grupo E04 - Agentifai

Braga, 29 de novembro de 2024

Resumo

No âmbito da Unidade Curricular Projeto de Informática, do segundo ano do Mestrado em Engenharia Informática, e em colaboração com a empresa Agentifai, foi desenvolvido um software destinado à síntese de voz em tempo real. O objetivo deste projeto consiste em criar um sistema capaz de gerar voz a partir de texto. A solução permitirá a personalização da voz e funcionará de forma independente de terceiros. Este sistema poderá ser integrado noutros sistemas. O presente documento tem como finalidade documentar todo o desenvolvimento técnico do projeto.

Conteúdo

1 Estrutura do Projeto 4
2 Modelo 4
3 Protocolo de Streaming 5
3.1 Protofile 5
3.2 Compilação 6
3.3 Implementar servidor gRPC 7
3.4 Implementar cliente gRPC 8
4 Containerização 8
4.1 Estrutura Geral 8
4.2 Gestão dos Containers com Docker Compose 8
4.3 Rede e Comunicação 8
4.4 Considerações finais 9
5 Normalização 9
5.1 Principais Funcionalidades 9
5.2 Arquitetura 10
5.3 Detalhes de Implementação 10
5.4 Regras 10
5.4.1 Gestão de Pontuação 10
5.4.2 Conversão de Números 11
5.4.3 Conversão de Datas 11
5.4.4 Substituições Personalizadas 12
5.4.5 Abreviaturas 12
5.4.6 Configuração 12
5.5 Fluxo 13
A Anexo 13
Anexo 1 Estrutura do projeto 13

1 Estrutura do Projeto

Nos anexos está presente a estrutura do projeto.

Em baixo está explicada a estrutura:

- *app*: Código relativo ao frontend da aplicação
- *app_api*: Ficheiros relativos à API.
- *app_client*: Código relativo a um cliente_gRPC.
- *app_server*: Código relativo ao servidor_gRPC.
- *app_normalizer*: Ficheiros relativos à normalização do texto
- *app_proxy*: Código relativo ao servidor proxy.
- *config*: Tem os ficheiros de configuração para o modelo e à normalização
- *db*: Ficheiros responsáveis pela criação e população da base de dados.
- *docker*: Tem os dockerfiles.
- *environments*: Ficheiros com todas as dependências do servidor, cliente, proxy e normalizador.
- *grpcs*: Local aonde estão os protofile
- *inputs*: Local aonde estão guardadas as vozes para serem usadas para a geração de audio.
- *outputs*: Local aonde ficam guardados os audios gerados.
- *ssml_parser*: Local aonde está o código relacionado ao parsing de ficheiros ssml.

2 Modelo

Foi escolhida a biblioteca *opensource* **Coqui TTS**, que se destaca pela sua abrangência e flexibilidade. O Coqui TTS é mais completo em comparação com outras bibliotecas, ao integrar uma variedade maior de modelos pré--treinados, como, por exemplo, o *Tacotron2* e o *FastSpeech2*. Esta diversidade de modelos permite ajustes mais precisos e a possibilidade de alcançar resultados de maior qualidade na conversão de texto para voz.

Adicionalmente, foi utilizado o *XTTS*, um modelo de geração de voz baseado nas tecnologias do Coqui TTS, que oferece a funcionalidade avançada de clonagem de vozes, inclusive para outras línguas. Esta capacidade é particularmente útil, ao permitir replicar características específicas de uma voz, economizando tempo no processo de treino e facilitando a adaptação do modelo a diferentes necessidades linguísticas e vocais.

Para treino, foram utilizados datasets constituídos por audiobooks e tedtalks. Uma característica importante para a escolha dos dados, para além de possuir áudios, foi possuir as legendas dos mesmos. Além disso, dada a dimensão do modelo, foi utilizado o Colab Pro.

Para customização do modelo, conseguimos controlar os seguintes parâmetros:

- length_penalty: Controla a preferência de outputs mais longos ou curtos.
- Repetition_penalty: Desencoraja a repetição de texto no áudio gerado
- tok_k: Limita o espaço de amostragens às k opções mais prováveis.
- top_p: Ativa a amostragem de núcleo, selecionando tokens do subconjunto mais provável.
- temperature: ajusta o nível de aleatoriedade no processo de geração.

Para correr o modelo standalone, basta correr o ficheiro ***intlex.py*** que está na base do projeto com o seguinte comando:

```bash
python intlex.py [TEXT] [CONFIG] [--args]
```

Para treinar basta correr o notebook ***Train.ipynb***

3 Protocolo de Streaming

Para protocolo de streaming utilizamos os gRPCs (Google Remote Procedure Call). Tal como o nome indica, é baseado em RPC - remote procedure call - em que consiste nos clientes chamarem métodos remotos no servidor como se estivessem chamando métodos locais. Utiliza HTTP/2 como protocolo de transporte o que permite ter, por exemplo, multiplexagem de streams.

Um dos pontos fortes que nos fez escolher este protocolo comparativamente a, por exemplo, Websockets, foi a sua alta eficiência e escalabilidade. De seguida, o facto de ter um grande suporte multilíngua permitiu assim termos maior flexibilidade de escolher a linguagem para desenvolver o servidor e o cliente.

De seguida estão descritos algumas peças e procedimentos fundamentais quando programamos gRPCs.

3.1 Protofile

O protofile (.proto) é uma componente essencial nos gRPCs. É nele que é definida a estrutura das mensagens e serviços que vão ser usados na comunicação entre o cliente e o servidor.

O protofile é escrito usando a linguagem **Protocol Buffers (Protobuf)** que tem como mecanismo a serialização e deserialização.

Exemplo:

Em baixo temos um exemplo de um protofile seguido da sua explicação e passos essenciais para a sua definição.

```proto
syntax = "proto3";

service NormalizerService {
  rpc Normalize (NormalizeRequest) returns (NormalizeResponse);
}

message NormalizeRequest {
  string text = 1; // Text to be normalized
}

message NormalizeResponse {
  string normalized_text = 1; // Normalized text result
}
```

1. **Sintaxe e Versão Protobuf**

Na linha 1 do exemplo definimos de imediato a sintaxe e a sua versão, neste caso *proto3*, utilizada no ficheiro.

2. **Definição de um serviço**

Entre as linhas 3 a 5 definimos um serviço chamado *NormalizeService*.

Os serviços são responsáveis por definir a interface dos gRPCs. Os métodos do serviço aceitam uma mensagem de dado tipo e devolvem a mensagem de um outro certo tipo, no exemplo dado, o método *normalize* aceita uma mensagem do tipo *NormalizeRequest* e devolve uma mensagem do tipo *NormalizeResponse*.

3. **Definição das Mensagens**

Nas definições das mensagens são definidos os dados que o cliente recebe ou envia ao servidor.

No exemplo, a mensagem *NormalizeRequest* define os dados (texto) que o cliente envia ao servidor e a mensagem *NormalizeResponse*, tal como o nome indica, define os dados (mesmo texto, só que normalizado) que o servidor envia de volta ao cliente após o processamento. O fluxo da comunicação entre o cliente e o servidor baseia-se na base da solicitação e resposta.

3.2 Compilação

Depois de termos o protofile configurado é necessário compilar o ficheiro de forma a gerar as classes e os métodos do gRPC na língua que se vai trabalhar. Os gRPCs têm suporte para várias linguagens, sendo exemplo delas Python, Go, Java, entre outras.

No nosso caso, dado que foi utilizado Python, para compilar é necessário correr o seguinte comando:

```bash
protoc --python_out=./ --grpc_python_out=./ <protofileName>.proto
```

Este comando vai gerar dois ficheiros:

- **your_protofile_pb2.py**: Contem as classes Python para as mensagens definidas no protofile.
- **your_protofile_pb2_grpc.py**: Contem código gRPC específico. Define a classe *Stub* (lado do cliente) e a classe *Servicer* (lado do servidor).

3.3 Implementar servidor gRPC

O servidor é responsável por implementar a lógica do serviço definida no protofile. Através do código gerado pelo protoc, podemos criar o servidor que recebe as chamadas do cliente e responde de acordo com a lógica de negócio.

```python
class NormalizerService(normalizer_pb2_grpc.NormalizerServiceServicer):
    def __init__(self, args):
        self.normalizer = Normalizer(args.rules)

    def Normalize(self, request, context):
        normalized_text = self.normalizer.normalize_text(request.text.strip())
        print(f"Received text: '{request.text}'")
        print(f"Normalized text: '{normalized_text}'")
        return normalizer_pb2.NormalizeResponse(normalized_text=normalized_text)

def serve(args):
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    normalizer_pb2_grpc.add_NormalizerServiceServicer_to_server(NormalizerServic
    server)
    server.add_insecure_port(f'[::]:{NORMALIZER_PORT}')
    print(f"Normalizer server running on port {NORMALIZER_PORT}...")
    server.start()
    server.wait_for_termination()

if __name__ == "__main__":
    serve()
```

No exemplo, a classe *NormalizerService* implementa o serviço definido e dentro desta temos a implementação do método (*Normalize*) definido no protofile.

3.4 Implementar cliente gRPC

O cliente é a entidade responsável por fazer chamadas ao servidor gRPC, através da requisição e recessão de respostas. O código gerado pelo protoc também inclui o código necessário para criar o cliente.

4 Containerização

Nesta seção é detalhado o uso do Docker para orquestrar os diferentes serviços do projeto de síntese de voz em tempo real (TTS). O projeto é composto por múltiplos componentes que interagem entre si, cada um encapsulado num container independente. A comunicação entre os serviços é facilitada pelo uso de uma rede Docker configurada com o driver bridge.

4.1 Estrutura Geral

A solução está dividida nos seguintes componentes:

1. Frontend: Interface do utilizador para interação com o sistema.
2. Client: Responsável por intermediar a comunicação entre o frontend e o restante do sistema.
3. API: Faz a gestão das solicitações e responde ao client, conectando ao proxy.
4. Proxy: Facilita a comunicação entre a API e o servidor principal.
5. Server: Onde o modelo TTS é executado para gerar os áudios.

Todos os serviços são definidos em Dockerfiles individuais e configurados através do ficheiro docker-compose.

4.2 Gestão dos Containers com Docker Compose

O ficheiro docker-compose é responsável por gerir os serviços descritos anteriormente. Os principais pontos de configuração incluem:

- Versão: O ficheiro utiliza a versão 3.8 do Docker Compose.
- Rede: Todos os serviços estão conectados na rede tts_network com driver bridge.
- Dependências: Cada serviço especifica os serviços dos quais depende para garantir que sejam iniciados na ordem correta.
- Exemplo: O Frontend depende do API, e este, por sua vez, depende do Proxy.

4.3 Rede e Comunicação

Os serviços utilizam uma rede única tts_network, garantindo que:

- Todos os serviços conseguem comunicar entre si através de nomes de container (DNS interno do Docker).
- As portas necessárias são expostas apenas para comunicação interna.

4.4 Considerações finais

- Portabilidade: O uso do Docker permite que o projeto seja executado em qualquer sistema com Docker instalado, com configuração mínima.
- Escalabilidade: A arquitetura modular permite adicionar novas funcionalidades ou instâncias de serviços sem alterar o sistema existente.
- Segurança: Apenas os serviços essenciais têm portas expostas externamente. A comunicação interna ocorre na rede isolada tts_network.

Este design modular com Docker assegura uma integração eficiente entre os componentes, garantindo que o projeto de síntese de voz em tempo real funcione de forma consistente.

5 Normalização

O Serviço de Normalização é um microserviço responsável por pré-processar entradas de texto, garantindo um formato padronizado e consistente antes de serem enviadas para o processo de síntese de voz. Utiliza um *design* modular, permitindo flexibilidade na gestão de transformações de texto, como ajustes de pontuação, conversão de números em palavras, formatação de datas e expansão de abreviaturas.

Este serviço suporta múltiplos idiomas e é configurável através de um ficheiro (`rules.toml`), permitindo uma adaptação simples a novos casos de uso ou idiomas.

5.1 Principais Funcionalidades

- Normalização de Texto:
  - Converter texto bruto num formato previsível adequado para síntese. Responsável por gerir diversos elementos textuais, como pontuação, números e abreviaturas.
- Suporte a Múltiplos Idiomas:
  - Adaptação de regras de normalização com base no idioma (ex.: *pt* para Português, *en* para Inglês).
- Comunicação via gRPC:
  - Disponibiliza o método *Normalize* para processar pedidos de texto provenientes do servidor responsável pela síntese de voz.
  - Retorna uma *NormalizeResponse* com o texto transformado.
- Modularidade:
  - Regras implementadas como módulos independentes. Ativação/ desativação de regras através do ficheiro de configuração.

5.2 Arquitetura

- Lógica Principal:
  - A classe *Normalizer* é o ponto de entrada para aplicar as regras de normalização.
  - As regras são carregadas dinamicamente com base no ficheiro de configuração.
- Módulos de Regras:
  - Cada regra é implementada como uma classe separada que herda da classe base *NormalizationRule*.
- Serviço *gRPC*:
  - Ficheiro: normalizer.py
  - Define o serviço *NormalizerService* e gere os pedidos.

5.3 Detalhes de Implementação

Ficheiro: `normalizer.py`

O *NormalizerService* inicializa a class *Normalizer* com uma configuração de regras e expõe o método Normalize:

```python
class NormalizerService(normalizer_pb2_grpc.NormalizerServiceServicer):
    def __init__(self, args):
        if args.rules is None:
            args.rules = NORMALIZER_RULES
        self.normalizer = Normalizer(args.rules)

    def Normalize(self, request, context):
        normalized_text = self.normalizer.normalize_text(request.text.strip())
        return normalizer_pb2.NormalizeResponse(normalized_text=normalized_text)

```

O serviço escuta numa porta configurável (*NORMALIZER_PORT*) e comunica com outros componentes através de *gRPC*.

5.4 Regras

5.4.1 Gestão de Pontuação

Ficheiro: `punctuation_handling.py`

Funcionalidade:

- Remove ou substitui pontuação especificada.
- Reduz múltiplos espaços consecutivos para um único espaço.

Exemplo:

```json
Entrada: "Olá, mundo!"
Saída: "Olá mundo"
```

- Parâmetros de Configuração:

```toml
[rules.punctuation_handling]
enabled: Se a regra está ativa ou não
remove: Lista de pontuações a remover.
replace_with_space: Lista de pontuações a substituir por espaços
```

5.4.2 Conversão de Números

Ficheiro: `number_conversion.py`

Funcionalidade:

- Converte números em palavras.
- Gerir percentagens, ordinais, frações e intervalos.

Exemplo:

```json
Entrada: "25%" (idioma: pt)
Saída: "vinte e cinco por cento"
```

Parâmetros de Configuração:

```toml
  enabled: se a regra está ativa ou não.
  detect_percentage: Normalizar percentagens.
  detect_years: Normalizar anos.
  detect_ordinal: Detetar se um número é Ordinal.
  handle_fractions: Converte frações.
  handle_ranges: Converte "ranges" de números.
  handle_currency: Converte símbolos de moeda em palavras.
  supported_currency: Lista de moedas suportadas.
  currency_map: Ficheiro dicionário com as informações de cada moeda.
  type: Tipo de output dos números em geral.
```

5.4.3 Conversão de Datas

Ficheiro: `date_conversion.py`

Funcionalidade:

- Transforma formatos de datas em palavras.
- Suporta datas parciais (ex.: “Janeiro 2020”).

Exemplo:

```json
Entrada: "2022-12-25"
```

```
2 Saída: "vinte e cinco de dezembro de dois mil e vinte e dois"
```

Parâmetros de Configuração:

```toml
allowed_separator: Especifica separadores válidos para datas.
formats: Define formatos de data aceitáveis.
```

5.4.4 Substituições Personalizadas

Ficheiro: `custom_replacements.py`

Funcionalidade:

- Substitui padrões definidos num ficheiro JSON (custom_replacements.json).

Exemplo:

```json
Entrada: "IA é fascinante"
Saída: "Inteligência Artificial é fascinante"
```

Parâmetros de Configuração:

```toml
case_sensitive: Especifica se as substituições são sensíveis a maiúsculas/minúsculas.
use_regex_flags: Ativa funcionalidades avançadas de regex.
```

5.4.5 Abreviaturas

Ficheiro: `abbreviations.py`

Funcionalidade:

- Expande abreviaturas utilizando um mapeamento JSON (abbreviations.json).

Exemplo:

```json
Entrada: "Dr. Silva"
Saída: "Doutor Silva"
```

Parâmetros de Configuração:

```toml
abbreviation_file: Caminho para o ficheiro de mapeamento.
mapping: Mapeamentos de abreviaturas.
```

5.4.6 Configuração

Ficheiro: `rules.toml`

Permite ativar/desativar regras específicas e definir os seus parâmetros.
Exemplo:

```toml
[rule_order]
rules = ["punctuation_handling", "number_conversion", "date_conversion", "abbreviations"]

[punctuation_handling]
enabled = true
remove = ["!", "?"]
replace_with_space = [",", ";"]

[number_conversion]
enabled = true
handle_currency = true
detect_percentage = true
detect_ordinal = true

[date_conversion]
enabled = true
formats = ["yyyy-MM-dd", "MM/dd/yyyy"]
```

5.5 Fluxo

Entrada: `"Dr. Silva deve $25,50 desde 01/01/2022."`

Passos:

1. Expansão de Abreviaturas: `"Doutor Silva deve \$25,50 desde 01/01/2022."`
2. Conversão de Números: `"Doutor Silva deve vinte e cinco dólares e cinquenta cêntimos desde 01/01/2022."`
3. Conversão de Datas: `"Doutor Silva deve vinte e cinco dólares e cinquenta cêntimos desde o primeiro de janeiro de dois mil e vinte e dois."`

Saída: `"Doutor Silva deve vinte e cinco dólares e cinquenta cêntimos desde o primeiro de janeiro de dois mil e vinte e dois."`

A Anexo

Anexo 1 Estrutura do projeto

```bash
├── app
│   ├── package.json
│   ├── postcss.config.js
│   ├── public
│   ├── README.md
│   ├── src
│   └── tailwind.config.js
├── app_api
│   ├── __main__.py
```

```
10  │   ├── outputs
11  │   └── __pycache__
12  ├── app_client
13  │   ├── client.py
14  │   ├── __init__.py
15  │   ├── __main__.py
16  │   ├── __pycache__
17  │   ├── teste.wav
18  │   ├── tts_pb2_grpc.py
19  │   └── tts_pb2.py
20  ├── app_normalizer
21  │   ├── abbreviations.py
22  │   ├── base.py
23  │   ├── custom_replacements.py
24  │   ├── date_conversion.py
25  │   ├── __init__.py
26  │   ├── __main__.py
27  │   ├── normalizer.py
28  │   ├── number_conversion.py
29  │   ├── punctuation_handling.py
30  │   └── __pycache__
31  ├── app_proxy
32  │   ├── __main__.py
33  │   └── proxy.py
34  ├── app_server
35  │   ├── intlex.py
36  │   ├── __main__.py
37  │   ├── __pycache__
38  │   ├── tts_pb2_grpc.py
39  │   └── tts_pb2.py
40  ├── config
41  │   ├── intlex_config.json
42  │   ├── normalization
43  │   └── proxy_config.json
44  ├── db
45  │   ├── create_tables.py
46  │   ├── __init__.py
47  │   ├── models.py
48  │   ├── __pycache__
49  │   └── tts_system.db
50  ├── docker
51  │   ├── comandos.md
52  │   ├── Dockerfile.api
```

```
53  │   ├── Dockerfile.app
54  │   ├── Dockerfile.client
55  │   ├── Dockerfile.normalizer
56  │   ├── Dockerfile.proxy
57  │   └── Dockerfile.server
58  ├── docker-compose.yaml
59  ├── docs
60  │   ├── book.toml
61  │   └── src
62  ├── enviroments
63  │   ├── api_requirements.txt
64  │   ├── client_requirements.txt
65  │   ├── dev_requirements.txt
66  │   ├── normalizer_requirements.txt
67  │   ├── proxy_requirements.txt
68  │   └── server_requirements.txt
69  ├── grpcs
70  │   ├── generate_protos.sh
71  │   ├── normalizer_pb2_grpc.py
72  │   ├── normalizer_pb2.py
73  │   ├── proto
74  │   ├── __pycache__
75  │   ├── tts_pb2_grpc.py
76  │   └── tts_pb2.py
77  ├── inputs
78  │   ├── natural_text_examples
79  │   ├── ssml_examples
80  │   └── voices
81  ├── intlex.py
82  ├── model
83  │   ├── run
84  │   ├── xtts_v1_initial_parameters.py
85  │   ├── xtts_v1_pt_bad_parameters.py
86  │   ├── xtts_v2_pt_colab_pro_notebook.ipynb
87  │   └── xtts_v2_pt_colab_pro.py
88  ├── outputs
89  │   ├── colab_my_model.wav
90  │   ├── funny_song.wav
91  │   ├── generated.wav
92  │   ├── good_temp_0.1.wav
93  │   ├── morgan_freeman.wav
94  │   ├── my_model.wav
95  │   ├── wtf_cool_voice.wav
```

```
96  │   ├── xtts-ft_0.75_sem_pontuação.wav
97  │   ├── xtts-ft_0.75.wav
98  │   ├── xtts-ft_1_sem_pontuação.wav
99  │   ├── xtts-ft_1.wav
100 │   ├── xtts-ft.wav
101 │   └── your_tts.wav
102 ├── README.md
103 ├── run.sh
104 ├── scripts
105 │   ├── docker_ip.sh
106 │   └── download_model.sh
107 ├── ssml_parser
108 │   ├── convert_numbers.py
109 │   ├── expected_output.txt
110 │   ├── input2.xml
111 │   ├── input3.xml
112 │   ├── input.xml
113 │   ├── output2.txt
114 │   ├── output.txt
115 │   ├── teste2.go
116 │   └── teste.py
117 └── testing.sh
```